

Once `head(penguins)` runs, **send me what you get**. Don't move ahead yet—we'll use your actual output to figure out the next step.

Description:df [6 × 3]

	<code>.pred_female</code>	<code><dbl></code>	<code>.pred_class</code>	<code><chr></code>	<code>sex</code>	<code><chr></code>
1	0.9921746	female	female			
2	0.9542394	female	female			
3	0.9847350	female	female			
4	0.1870206	male	female			
5	0.9947012	female	female			
6	0.9999891	female	femal			

Great — this already teaches us something important about the dataset.

From your first six rows, we can see:

- `.pred_female` is a **number between 0 and 1** representing the model's predicted probability of the penguin being female.
- `.pred_class` is the model's classification using its default threshold.

- **sex** is the **actual observed class**.

For example, look at **row 4**:

```
.pred_female = 0.187
```

```
.pred_class = male
```

```
sex = female
```

The model only gave that penguin an **18.7% probability of being female**, so it predicted male. But the actual sex was female. That's an incorrect prediction. Later, we'll learn that this type of mistake has a specific name.

Step 2 — Check the class balance

Before evaluating the model, your assignment wants the **null error rate**.

But don't calculate it yet. First let's answer a simpler question:

How many actual females and males are in our dataset?

Add this underneath your previous work:

```
## Null Error Rate
```

Before evaluating the model, I first examined the distribution of the actual classes.

Then make another R chunk:

```
table(penguins$sex)
```

Why are we doing this?

Imagine our dataset had:

- 90 females
- 10 males

A completely dumb model could predict **"female" for every single penguin** and still be correct 90% of the time.

That's why accuracy alone can sometimes be misleading.

The **null model** basically says:

"I'm not going to learn anything. I'll just predict the most common class every time."

The **null error rate** tells us how often that strategy would be **wrong**.

Run:

```
table(penguins$sex)
```

Send me the result. Then I'll have you figure out the null error rate from those numbers rather than me just giving you the calculation.

Exactly. Now we know the **0.2 threshold** gives us:

- Accuracy = **91.4%**
- Precision = **86.0%**
- Recall = **94.9%**
- F1 = **90.2%**

Notice **recall is especially high**. That fits what we observed earlier: with a low threshold, the model is more willing to predict "**female**", so it catches **37 of the 39 actual females**. The tradeoff is that it also incorrectly labels some males as female, which hurts precision.

Step 11 — Calculate all three thresholds efficiently

We understand the formulas now, so there's no reason to manually repeat all four calculations twice more.

Let's create a small dataframe using the confusion-matrix values we already found:

```
metrics <- data.frame(  
  threshold = c(0.2, 0.5, 0.8),  
  TP = c(37, 36, 36),  
  FP = c(6, 3, 2),  
  TN = c(48, 51, 52),  
  FN = c(2, 3, 3)  
)
```

metrics

Run that first. You should see **three rows**, one for each threshold.

Then we'll have R calculate the four metrics from those columns instead of typing each formula three separate times.

Once that works, tell me and we'll add **Accuracy, Precision, Recall, and F1** to this table.

works

Good. Now we're going to let R calculate the metrics using the columns we just created.

Step 12 — Add the performance metrics

We could use a package that has built-in classification metrics, but since the assignment gives us the formulas and we're trying to understand them, calculating them ourselves makes the process clearer.

Add this chunk:

```
metrics$accuracy <- (metrics$TP + metrics$TN) /  
(metrics$TP + metrics$FP + metrics$TN + metrics$FN)
```

```
metrics$precision <- metrics$TP /  
(metrics$TP + metrics$FP)
```

```
metrics$recall <- metrics$TP /  
(metrics$TP + metrics$FN)
```

```
metrics$f1 <- 2 * (metrics$precision * metrics$recall) /  
(metrics$precision + metrics$recall)
```

metrics

What is different from what we did before?

Previously we had:

```
precision <- TP / (TP + FP)
```

That calculated precision for **one threshold**.

Now:

```
metrics$precision <- metrics$TP / (metrics$TP + metrics$FP)
```

tells R:

Take the TP and FP values from **each row** and calculate precision for each threshold.

So one piece of code handles all three thresholds.